

Lab Assignment 4

CMPE 146 Section 1

Professor Bhawandeep Singh Harsh

Andrew Bertch

Robert Trinh

Table of Contents

Exercise 1 – DMA.....	3
Exercise 1.1 – Speedup	4
Exercise 1.2 – Different data block sizes	5
Exercise 2 – Flash memory.....	14
Exercise 2.1 – String modifications.....	14
Exercise 2.2 – Program flash memory.....	16
Exercise 2.3 – Non-volatile power on reset counter	17
Appendix A – MCU Source Code.....	18
Speedup_DMA.c	18
diff_blocks.c	22
string_mod.c	27
prog_flash.c.....	28
power_counter.c	29

Exercise 1 – DMA

In this exercise, we will look at using direct memory access (DMA) to more efficiently pass data blocks to the CRC accelerator. DMA will allow the CRC accelerator to access the data needed instead of the using the CPU and main() method to pass data to the CRC accelerator.

Exercise 1.1 – Speedup

In this exercise, we will set up and use DMA with the CRC accelerator. We will make use of the program from exercise 2.2 in Lab 3 and add the DMA method. The time it takes for both methods will be measured and compared. The results are shown below with Time one corresponding to the previous hardware method and Time two corresponding to the new DMA method added in this exercise.

CORTEX_MoP: First four bytes: DC, 04, 65, AA

CORTEX_MoP: Last four bytes: B2, 8C, AF, 7C

CORTEX_MoP: Time one: 1448.531 us, Checksum one: 9991F447

CORTEX_MoP: Time two: 1053.562 us, Checksum two: 9991F447

CORTEX_MoP: Ratio of speedup: 1.375

Exercise 1.2 – Different data block sizes

In this exercise, we will expand on the previous exercise but add tests running the hardware and DMA methods with different block sizes of data. We will test with block sizes of 2, 4, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, and 10240 integers. The outputs of the program are shown below.

CORTEX_MoP: First four bytes: DC, 04, 65, AA

CORTEX_MoP: Last four bytes: B2, 8C, AF, 7C

CORTEX_MoP: Block size: 2

CORTEX_MoP: Time one: 1.688 us, Checksum one: 94C6A30A

CORTEX_MoP: Time two: 14.000 us, Checksum two: 94C6A30A

CORTEX_MoP: Ratio of speedup: 0.121

CORTEX_MoP: Block size: 4

CORTEX_MoP: Time one: 2.313 us, Checksum one: 9B435F21

CORTEX_MoP: Time two: 14.281 us, Checksum two: 9BB95BoA

CORTEX_MoP: Ratio of speedup: 0.162

CORTEX_MoP: Block size: 16

CORTEX_MoP: Time one: 6.188 us, Checksum one: 616843BC

CORTEX_MoP: Time two: 15.531 us, Checksum two: A3A5678E

CORTEX_MoP: Ratio of speedup: 0.398

CORTEX_MoP: Block size: 32

CORTEX_MoP: Time one: 11.375 us, Checksum one: 9CABA672

CORTEX_MoP: Time two: 17.125 us, Checksum two: 99DF17A0

CORTEX_MoP: Ratio of speedup: 0.664

CORTEX_MoP: Block size: 64

CORTEX_MoP: Time one: 21.688 us, Checksum one: 708ACEC1

CORTEX_MoP: Time two: 20.594 us, Checksum two: A3822C83

CORTEX_MoP: Ratio of speedup: 1.053

CORTEX_MoP: Block size: 128

CORTEX_MoP: Time one: 42.375 us, Checksum one: BE1D7469

CORTEX_MoP: Time two: 27.125 us, Checksum two: 977FECAD

CORTEX_MoP: Ratio of speedup: 1.562

CORTEX_MoP: Block size: 256

CORTEX_MoP: Time one: 83.688 us, Checksum one: C1529CA4

CORTEX_MoP: Time two: 40.531 us, Checksum two: 2D6DB36F

CORTEX_MoP: Ratio of speedup: 2.065

CORTEX_MoP: Block size: 786

CORTEX_MoP: Time one: 254.750 us, Checksum one: 0A36E9DF

CORTEX_MoP: Time two: 95.719 us, Checksum two: D43EEC61

CORTEX_MoP: Ratio of speedup: 2.661

CORTEX_MoP: Block size: 1024

CORTEX_MoP: Time one: 331.687 us, Checksum one: CB91A9C6

CORTEX_MoP: Time two: 120.531 us, Checksum two: 0A1AB52F

CORTEX_MoP: Ratio of speedup: 2.752

CORTEX_MoP: Block size: 2048

CORTEX_MoP: Time one: 662.375 us, Checksum one: 0FBF20EA

CORTEX_MoP: Time two: 227.125 us, Checksum two: 31FBFFEA

CORTEX_MoP: Ratio of speedup: 2.916

CORTEX_MoP: Block size: 4096

CORTEX_MoP: Time one: 1323.688 us, Checksum one: 616DF5E1

CORTEX_MoP: Time two: 440.594 us, Checksum two: 3C64C005

CORTEX_MoP: Ratio of speedup: 3.004

CORTEX_MoP: Block size: 8192

CORTEX_MoP: Time one: 2646.375 us, Checksum one: F491E3A8

CORTEX_MoP: Time two: 867.125 us, Checksum two: BCoAD1A6

CORTEX_MoP: Ratio of speedup: 3.052

CORTEX_MoP: Block size: 10240

CORTEX_MoP: Time one: 3307.688 us, Checksum one: 3AB99F05

CORTEX_MoP: Time two: 1080.531 us, Checksum two: 9991F447

CORTEX_MoP: Ratio of speedup: 3.061

CORTEX_MoP: First four bytes: DC, 04, 65, AA

CORTEX_MoP: Last four bytes: B2, 8C, AF, 7C

CORTEX_MoP: Block size: 2

CORTEX_MoP: Time one: 1.875 us, Checksum one: 94C6A30A

CORTEX_MoP: Time two: 14.031 us, Checksum two: 94C6A30A

CORTEX_MoP: Ratio of speedup: 0.134

CORTEX_MoP: Block size: 4

CORTEX_MoP: Time one: 2.500 us, Checksum one: 9BB95BoA

CORTEX_MoP: Time two: 14.281 us, Checksum two: 9BB95BoA

CORTEX_MoP: Ratio of speedup: 0.175

CORTEX_MoP: Block size: 16

CORTEX_MoP: Time one: 6.250 us, Checksum one: A3A5678E

CORTEX_MoP: Time two: 15.531 us, Checksum two: A3A5678E

CORTEX_MoP: Ratio of speedup: 0.402

CORTEX_MoP: Block size: 32

CORTEX_MoP: Time one: 11.250 us, Checksum one: 99DF17A0

CORTEX_MoP: Time two: 17.156 us, Checksum two: 99DF17A0

CORTEX_MoP: Ratio of speedup: 0.656

CORTEX_MoP: Block size: 64

CORTEX_MoP: Time one: 21.250 us, Checksum one: A3822C83

CORTEX_MoP: Time two: 20.625 us, Checksum two: A3822C83

CORTEX_MoP: Ratio of speedup: 1.030

CORTEX_MoP: Block size: 128

CORTEX_MoP: Time one: 41.250 us, Checksum one: 977FECAD

CORTEX_MoP: Time two: 27.156 us, Checksum two: 977FECAD

CORTEX_MoP: Ratio of speedup: 1.519

CORTEX_MoP: Block size: 256

CORTEX_MoP: Time one: 81.250 us, Checksum one: 2D6DB36F

CORTEX_MoP: Time two: 40.531 us, Checksum two: 2D6DB36F

CORTEX_MoP: Ratio of speedup: 2.005

CORTEX_MoP: Block size: 786

CORTEX_MoP: Time one: 246.781 us, Checksum one: D43EEC61

CORTEX_MoP: Time two: 95.750 us, Checksum two: D43EEC61

CORTEX_MoP: Ratio of speedup: 2.577

CORTEX_MoP: Block size: 1024

CORTEX_MoP: Time one: 321.250 us, Checksum one: 0A1AB52F

CORTEX_MoP: Time two: 120.531 us, Checksum two: 0A1AB52F

CORTEX_MoP: Ratio of speedup: 2.665

CORTEX_MoP: Block size: 2048

CORTEX_MoP: Time one: 641.250 us, Checksum one: 31FBFFEA

CORTEX_MoP: Time two: 227.156 us, Checksum two: 31FBFFEA

CORTEX_MoP: Ratio of speedup: 2.823

CORTEX_MoP: Block size: 4096

CORTEX_MoP: Time one: 1281.250 us, Checksum one: 3C64C005

CORTEX_MoP: Time two: 440.625 us, Checksum two: 3C64C005

CORTEX_MoP: Ratio of speedup: 2.908

CORTEX_MoP: Block size: 8192

CORTEX_MoP: Time one: 2561.250 us, Checksum one: BCoAD1A6

CORTEX_MoP: Time two: 867.156 us, Checksum two: BCoAD1A6

CORTEX_MoP: Ratio of speedup: 2.954

CORTEX_MoP: Block size: 10240

CORTEX_MoP: Time one: 3201.250 us, Checksum one: 9991F447

CORTEX_MoP: Time two: 1080.531 us, Checksum two: 9991F447

CORTEX_MoP: Ratio of speedup: 2.963

Comparing the ratio of speedup between data block sizes, it is clear that DMA is much faster than the hardware method with larger data blocks but can become slower with smaller data blocks. This is likely due to the MSPMoG350x having 7 DMA channels and when operating in single transfer mode, each transfer requires a transfer trigger. This results in significant overhead for each data transfer. However, we saw that starting at 64 integers being transferred that DMA caught up and was slightly faster. This is likely due to the DMA channels becoming saturated enough to be faster than this overhead and be faster than the hardware method.

Exercise 2 – Flash memory

In this exercise, we will look at how the compiler handles different parts of memory and do some flash memory programming.

Exercise 2.1 – String modifications

In this exercise, we run a provided program and observe how two different variables are handled when modified. Running the program results in the results shown below.

CORTEX_MoP: string1: xyz123

CORTEX_MoP: string2: xyz123

CORTEX_MoP: string1: xyz123

CORTEX_MoP: string2: oyz123

The first string doesn't get changed while the second string does get changed. This is because the first string is stored in the flash memory which with our current program cannot be modified. This means that when trying to change the 'x' to a 'o' nothing changes. The datasheet lists the flash memory as EEPROM. It also makes mention of a write mode that needs to be enabled to allow writing to the EEPROM. This can clearly be seen in figure 2.1 and 2.2. string1 is located in the flash memory and string2 is located in the SRAM. This can be seen in Table 1-1 in section 1.3 Platform Memory Map of MSPMo G-series datasheet from TI.

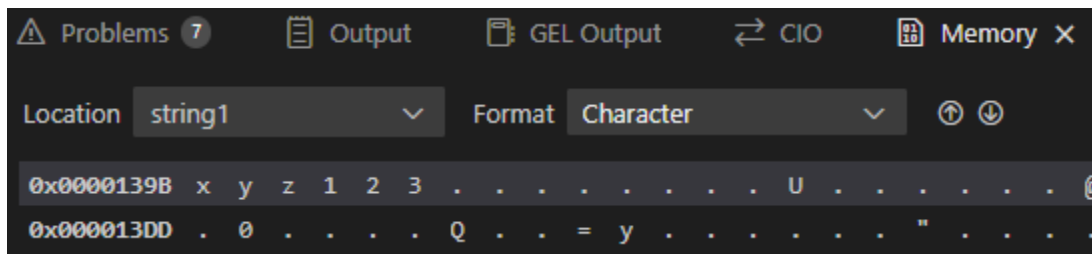


Figure 2.1

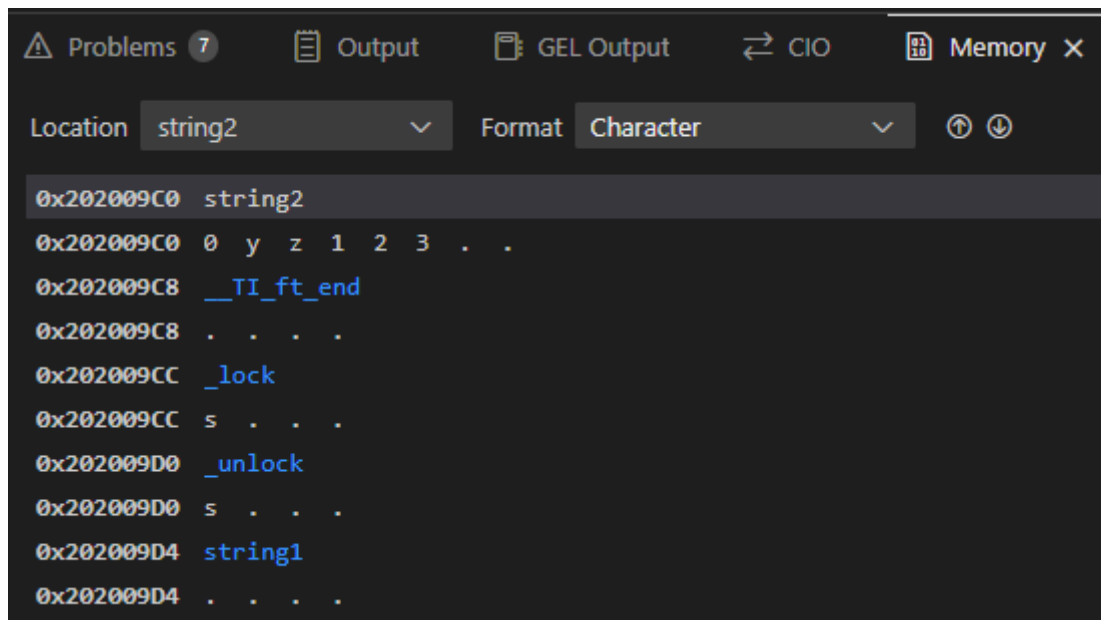


Figure 2.2

Exercise 2.2 – Program flash memory

In this exercise, we will learn to program the flash memory on our device. We learned that programming flash memory is much more involved than using RAM. We learn that we can only write 0's to the flash memory and cannot write 1's. The first step we must take is to unprotect the flash memory so that we can write to it. The relevant code to do that is included below.

```
DL_FlashCTL_unprotectSector(FLASHCTL, (uint32_t)flash_data,  
DL_FLASHCTL_REGION_SELECT_MAIN);
```

This one line is used before a write operation since the write operation will protect the memory section when completed. Since we are doing 8 writes, we use a loop and have the line above before each and every write operation. Not doing so will cause any future writes to not change the flash memory. The outputs of the whole program are included below.

```
CORTEX_MoP: new data: FE      write success  flash data FE 00 00 00 00 00 00 00 00
```

```
CORTEX_MoP: new data: FC      write success  flash data FC 00 00 00 00 00 00 00 00
```

```
CORTEX_MoP: new data: F8      write success  flash data F8 00 00 00 00 00 00 00 00
```

```
CORTEX_MoP: new data: Fo      write success  flash data Fo 00 00 00 00 00 00 00 00
```

```
CORTEX_MoP: new data: Eo      write success  flash data Eo 00 00 00 00 00 00 00 00
```

```
CORTEX_MoP: new data: Co      write success  flash data Co 00 00 00 00 00 00 00 00
```

```
CORTEX_MoP: new data: 8o      write success  flash data 8o 00 00 00 00 00 00 00 00
```

```
CORTEX_MoP: new data: oo      write success  flash data oo 00 00 00 00 00 00 00 00
```

Exercise 2.3 – Non-volatile power on reset counter

In this exercise, we will write a program that limits the amount of times a user can use the device. We will have a counter that is stored in the non-volatile memory to check how many times the device has been used and then disable use when the counter reaches the use threshold. Since this counter is stored in the flash memory, it will stay even if the device were to lose power or be turned off. The code to manipulate the counter is shown below.

```
new_data = aliasRegion[0] << 1;

DL_FlashCTL_unprotectSector(FLASHCTL, (uint32_t)flash_data,
DL_FLASHCTL_REGION_SELECT_MAIN);

DL_FlashCTL_programMemoryFromRAM8(FLASHCTL, (uint32_t)flash_data,
&new_data);
```

A video demonstrating our implementation of this counter and disable program is included [here](#).

Appendix A – MCU Source Code

Speedup_DMA.c

```
#include "ti_msp_dl_config.h"

#include <stdio.h>

#include <stdlib.h>

#define CRC32_SEED 0xFFFFFFFF

volatile bool dma_done;

void DMA_IRQHandler(void)
{
    switch (DL_DMA_getPendingInterrupt(DMA)) {
        case DL_DMA_EVENT_IIDX_DMACH0:
            dma_done = true;
            break;
        default:
            break;
    }
}

int main(void)
{
    SYSCFG_DL_init();

    DL_GPIO_enablePower(GPIOB);
```

```
DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINCM49,  
IOMUX_PINCM49_PF_GPIOB_DIO21, DL_GPIO_INVERSION_DISABLE,  
DL_GPIO_RESISTOR_PULL_UP, DL_GPIO_HYSTERESIS_DISABLE,  
DL_GPIO_WAKEUP_DISABLE);
```

```
// Set up to use the 32-bit counter TIMG12
```

```
DL_Timer_enablePower(TIMG12);
```

```
DL_Timer_ClockConfig config;
```

```
config.clockSel = DL_TIMER_CLOCK_BUSCLK;
```

```
config.divideRatio = DL_TIMER_CLOCK_DIVIDE_1;
```

```
config.prescale = 0;
```

```
DL_Timer_setClockConfig(TIMG12, &config);
```

```
DL_Timer_TimerConfig timerConfig;
```

```
timerConfig.timerMode = DL_TIMER_TIMER_MODE_PERIODIC_UP;
```

```
timerConfig.period = -1;
```

```
timerConfig.startTimer = DL_TIMER_START;
```

```
timerConfig.genIntermInt = DL_TIMER_INTERM_INT_DISABLED;
```

```
timerConfig.counterVal = 0;
```

```
DL_Timer_initTimerMode(TIMG12, &timerConfig);
```

```
// CRC setup
```

```
DL_CRC_reset(CRC);
```

```
DL_CRC_enablePower(CRC);
```

```
delay_cycles(POWER_STARTUP_DELAY);
```

```
DL_CRC_init(CRC, DL_CRC_32_POLYNOMIAL, DL_CRC_BIT_REVERSED,
```

```
DL_CRC_INPUT_ENDIANESS_LITTLE_ENDIAN,  
DL_CRC_OUTPUT_BYTESWAP_DISABLED);
```

```
DL_CRC_setSeed32(CRC, CRC32_SEED);
```

```

//DMA setup
#define DMA_CH0_CHAN_ID (0)
static const DL_DMA_Config gDMA_CH0Config = {
    .transferMode = DL_DMA_SINGLE_BLOCK_TRANSFER_MODE,
    .extendedMode = DL_DMA_NORMAL_MODE,
    .destIncrement = DL_DMA_ADDR_UNCHANGED,
    .srcIncrement = DL_DMA_ADDR_INCREMENT,
    .destWidth = DL_DMA_WIDTH_BYTE,
    .srcWidth = DL_DMA_WIDTH_BYTE,
    .trigger = DMA_SOFTWARE_TRIG,
    .triggerType = DL_DMA_TRIGGER_TYPE_EXTERNAL,
};
DL_DMA_clearInterruptStatus(DMA, DL_DMA_INTERRUPT_CHANNEL0);
DL_DMA_enableInterrupt(DMA, DL_DMA_INTERRUPT_CHANNEL0);
DL_DMA_initChannel(DMA, DMA_CH0_CHAN_ID , (DL_DMA_Config *)
&gDMA_CH0Config);
NVIC_EnableIRQ(DMA_INT_IRQn);

uint32_t start_value_one, start_value_two;
uint32_t end_value_one, end_value_two;
double time_one, time_two;

static uint8_t myData[10240];

srand(12345);
for(int i = 0; i < 10240; i++)

```

```

{
    myData[i] = rand();
}

printf("First four bytes: %02X, %02X, %02X, %02X\n", myData[0],
myData[1], myData[2], myData[3]);

printf("Last four bytes: %02X, %02X, %02X, %02X\n", myData[10236],
myData[10237], myData[10238], myData[10239]);

start_value_one = DL_Timer_getTimerCount(TIMG12);
for(int i = 0; i < 10240; i++)
{
    DL_CRC_feedData8(CRC, myData[i]);
}
uint32_t checksum_one = DL_CRC_getResult32(CRC);
end_value_one = DL_Timer_getTimerCount(TIMG12);
time_one = ((double)end_value_one - start_value_one) / (32000000)
* 1000000;

start_value_two = DL_Timer_getTimerCount(TIMG12);
DL_DMA_setSrcAddr(DMA, DMA_CH0_CHAN_ID, (uint32_t)&myData[0]);
DL_DMA_setDestAddr(DMA, DMA_CH0_CHAN_ID,
DL_CRC_getCRCINAddr(CRC));
DL_DMA_setTransferSize(DMA, DMA_CH0_CHAN_ID, 10240);
DL_DMA_enableChannel(DMA, DMA_CH0_CHAN_ID);
DL_CRC_setSeed32(CRC, CRC32_SEED);
dma_done = false;
DL_DMA_startTransfer(DMA, DMA_CH0_CHAN_ID);

```

```

while(!dma_done);

uint32_t checksum_two = DL_CRC_getResult32(CRC);

end_value_two = DL_Timer_getTimerCount(TIMG12);

time_two = ((double)end_value_two - start_value_two) / (32000000)
* 1000000;

printf("Time one: %.3f us, Checksum one: %08X\n", time_one,
checksum_one);

printf("Time two: %.3f us, Checksum two: %08X\n", time_two,
checksum_two);

double ratio = time_one/time_two;

printf("Ratio of speedup: %.3f\n", ratio);

while (1) {

}

}

```

diff_blocks.c

```

#include "ti_msp_dl_config.h"

#include <stdio.h>

#include <stdlib.h>

#define CRC32_SEED 0xFFFFFFFF

volatile bool dma_done;

int size_array[] = {2, 4, 16, 32, 64, 128, 256, 786, 1024, 2048, 4096,
8192, 10240};

```

```

void DMA_IRQHandler(void)
{
    switch (DL_DMA_getPendingInterrupt(DMA)) {
        case DL_DMA_EVENT_IIDX_DMACH0:
            dma_done = true;
            break;
        default:
            break;
    }
}

int main(void)
{
    SYSCFG_DL_init();

    DL_GPIO_enablePower(GPIOB);

    DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINCM49,
    IOMUX_PINCM49_PF_GPIOB_DIO21, DL_GPIO_INVERSION_DISABLE,
    DL_GPIO_RESISTOR_PULL_UP, DL_GPIO_HYSTERESIS_DISABLE,
    DL_GPIO_WAKEUP_DISABLE);

    // Set up to use the 32-bit counter TIMG12
    DL_Timer_enablePower(TIMG12);
    DL_Timer_ClockConfig config;
    config.clockSel = DL_TIMER_CLOCK_BUSCLK;
    config.divideRatio = DL_TIMER_CLOCK_DIVIDE_1;
    config.prescale = 0;

```

```

DL_Timer_setClockConfig(TIMG12, &config);
DL_Timer_TimerConfig timerConfig;
timerConfig.timerMode = DL_TIMER_TIMER_MODE_PERIODIC_UP;
timerConfig.period = -1;
timerConfig.startTimer = DL_TIMER_START;
timerConfig.genIntermInt = DL_TIMER_INTERM_INT_DISABLED;
timerConfig.counterVal = 0;
DL_Timer_initTimerMode(TIMG12, &timerConfig);

// CRC setup
DL_CRC_reset(CRC);
DL_CRC_enablePower(CRC);
delay_cycles(POWER_STARTUP_DELAY);
DL_CRC_init(CRC, DL_CRC_32_POLYNOMIAL, DL_CRC_BIT_REVERSED,
DL_CRC_INPUT_ENDIANESS_LITTLE_ENDIAN,
DL_CRC_OUTPUT_BYTESWAP_DISABLED);
DL_CRC_setSeed32(CRC, CRC32_SEED);

//DMA setup
#define DMA_CH0_CHAN_ID (0)
static const DL_DMA_Config gDMA_CH0Config = {
    .transferMode = DL_DMA_SINGLE_BLOCK_TRANSFER_MODE,
    .extendedMode = DL_DMA_NORMAL_MODE,
    .destIncrement = DL_DMA_ADDR_UNCHANGED,
    .srcIncrement = DL_DMA_ADDR_INCREMENT,
    .destWidth = DL_DMA_WIDTH_BYTE,
    .srcWidth = DL_DMA_WIDTH_BYTE,

```

```

        .trigger = DMA_SOFTWARE_TRIG,
        .triggerType = DL_DMA_TRIGGER_TYPE_EXTERNAL,
    };

    DL_DMA_clearInterruptStatus(DMA, DL_DMA_INTERRUPT_CHANNEL0);
    DL_DMA_enableInterrupt(DMA, DL_DMA_INTERRUPT_CHANNEL0);
    DL_DMA_initChannel(DMA, DMA_CH0_CHAN_ID , (DL_DMA_Config *)
&gDMA_CH0Config);
    NVIC_EnableIRQ(DMA_INT_IRQn);


    uint32_t start_value_one, start_value_two;
    uint32_t end_value_one, end_value_two;
    double time_one, time_two;


    static uint8_t myData[10240];


    srand(12345);
    for(int i = 0; i < 10240; i++)
    {
        myData[i] = rand();
    }


    printf("First four bytes: %02X, %02X, %02X, %02X\n", myData[0],
myData[1], myData[2], myData[3]);

    printf("Last four bytes: %02X, %02X, %02X, %02X\n", myData[10236],
myData[10237], myData[10238], myData[10239]);


    for (int i = 0; i < 13; i++) {
        start_value_one = DL_Timer_getTimerCount(TIMG12);

```

```

DL_CRC_setSeed32(CRC, CRC32_SEED);
for(int j = 0; j < size_array[i]; j++)
{
    DL_CRC_feedData8(CRC, myData[j]);
}
uint32_t checksum_one = DL_CRC_getResult32(CRC);
end_value_one = DL_Timer_getTimerCount(TIMG12);
time_one = ((double)end_value_one - start_value_one) /
(32000000) * 1000000;

start_value_two = DL_Timer_getTimerCount(TIMG12);
DL_DMA_setSrcAddr(DMA, DMA_CH0_CHAN_ID, (uint32_t)&myData[0]);
DL_DMA_setDestAddr(DMA, DMA_CH0_CHAN_ID,
DL_CRC_getCRCINAddr(CRC));
DL_DMA_setTransferSize(DMA, DMA_CH0_CHAN_ID, size_array[i]);
DL_DMA_enableChannel(DMA, DMA_CH0_CHAN_ID);
DL_CRC_setSeed32(CRC, CRC32_SEED);
dma_done = false;
DL_DMA_startTransfer(DMA, DMA_CH0_CHAN_ID);
while(!dma_done);
uint32_t checksum_two = DL_CRC_getResult32(CRC);
end_value_two = DL_Timer_getTimerCount(TIMG12);
time_two = ((double)end_value_two - start_value_two) /
(32000000) * 1000000;

printf("Block size: %d\n", size_array[i]);

```

```
    printf("Time one: %.3f us, Checksum one: %08X\n", time_one,
checksum_one);
```

```
    printf("Time two: %.3f us, Checksum two: %08X\n", time_two,
checksum_two);
```

```
    double ratio = time_one/time_two;
```

```
    printf("Ratio of speedup: %.3f\n", ratio);
```

```
}
```

```
while (1) {
```

```
}
```

```
}
```

string_mod.c

```
#include "ti_msp_dl_config.h"
```

```
#include <stdint.h>
```

```
#include <stdbool.h>
```

```
#include <stdio.h>
```

```
char* string1 = "xyz123";
```

```
char string2[] = "xyz123";
```

```
void main(void)
```

```
{
```

```
    SYSCFG_DL_init();
```

```
    printf("string1: %s\n", string1);
```

```

    printf("string2: %s\n", string2);
    string1[0] = '0';
    string2[0] = '0';
    printf("string1: %s\n", string1);
    printf("string2: %s\n", string2);
}

```

prog_flash.c

```
#include "ti_msp_dl_config.h"
```

```
#include <stdint.h>
```

```
#include <stdbool.h>
```

```
#include <stdio.h>
```

```
_Alignas(DL_FLASHCTL_SECTOR_SIZE) const uint8_t flash_data[] = {0xff};
```

```
void main(void)
```

```
{
```

```
    SYSCFG_DL_init();
```

```
    uint8_t new_data = 0xFF;
```

```
    for(int i = 0; i < 8; i++)
```

```
    {
```

```
        new_data &= ~(1 << i);
```

```
        printf("new data: %02X\t", new_data);
```

```
        DL_FlashCTL_unprotectSector(FLASHCTL, (uint32_t)flash_data,
DL_FLASHCTL_REGION_SELECT_MAIN);
```

```

        if(DL_FlashCTL_programMemoryFromRAM8(FLASHCTL,
(uint32_t)flash_data, &new_data)) {
            printf("write success\t");
        }

        uint8_t* aliasRegion = (uint8_t*)(0x41400000 +
((uint32_t)flash_data - 0x41000000));

        printf("flash data ");
        for(int j = 0; j < 8; j++)
        {
            printf("%02X ", aliasRegion[j]);
        }
        printf("\n");
    }

    while (1)
    {

    }
}

```

power_counter.c

```

#include "ti_msp_dl_config.h"
#include <stdint.h>
#include <stdbool.h>
#include <stdio.h>

```

```
_Alignas(DL_FLASHCTL_SECTOR_SIZE) const uint8_t flash_data[] = {0xff};
```

```
void main(void)
```

```
{
```

```
    SYSCFG_DL_init();
```

```
    DL_GPIO_enablePower(GPIOB);
```

```
    DL_GPIO_initDigitalOutput(IOMUX_PINCM57); // PB26, red LED in LED2
```

```
    DL_GPIO_initDigitalOutput(IOMUX_PINCM58); // PB27, green LED in  
LED2
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM57); // PB26, red LED
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM58); // PB27, green LED
```

```
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_26); // PB26, red LED
```

```
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_27); // PB27, green LED
```

```
    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_26);
```

```
    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_27);
```

```
    uint8_t* aliasRegion = (uint8_t*)(0x41400000 +  
((uint32_t)flash_data - 0x41000000));
```

```
    uint8_t new_data = aliasRegion[0];
```

```
    if(aliasRegion[0] == 0xF8)
```

```
    {
```

```
        //turn on red led
```

```

        DL_GPIO_setPins(GPIOB, DL_GPIO_PIN_26);
    }
    else
    {
        new_data = aliasRegion[0] << 1;

        DL_FlashCTL_unprotectSector(FLASHCTL, (uint32_t)flash_data,
DL_FLASHCTL_REGION_SELECT_MAIN);

        DL_FlashCTL_programMemoryFromRAM8(FLASHCTL,
(uint32_t)flash_data, &new_data);

        //blink green led
        while (1)
        {
            DL_GPIO_setPins(GPIOB, DL_GPIO_PIN_27);
            delay_cycles(24000000);
            DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_27);
            delay_cycles(24000000);
        }
    }
}

}

```